



AI VIET NAM – AI Conquer 2026

Tank Tank Bot: Hệ thống RAG hỗ trợ tra cứu tài liệu học tập AIO 2026

Nguyễn Xuân Thành

I. Tổng quan báo cáo

Báo cáo này trình bày quá trình xây dựng và nâng cấp **Tank Tank Bot**, một hệ thống hỏi đáp tài liệu học tập AIO 2026 theo hướng retrieval-augmented generation (RAG). Mục tiêu của project là hỗ trợ người học tra cứu nội dung từ tài liệu PDF và Notion, sinh câu trả lời tiếng Việt có căn cứ, đồng thời hiển thị nguồn trích dẫn để người dùng có thể kiểm chứng.

Project được triển khai theo hướng ứng dụng. Baseline ban đầu thể hiện luồng tối thiểu của một chatbot RAG trên PDF: upload tài liệu, trích xuất văn bản, chia đoạn, embedding, lưu vector, truy xuất ngữ cảnh và sinh câu trả lời. Bản nâng cấp mở rộng hệ thống theo nhiều hướng: bổ sung nguồn dữ liệu Notion, cải thiện trích xuất PDF, chunking có nhận thức cấu trúc, hybrid retrieval, reranking, MMR, citation, kiểm thử tự động và đánh giá offline.

Kết quả đánh giá trên bộ test grounded cho thấy pipeline nâng cấp **Hybrid RRF + Cohere Rerank + MMR** cải thiện rõ rệt chất lượng truy xuất so với vector-only. Cụ thể, context precision tăng từ 0.972 lên 1.000, context recall tăng từ 0.750 lên 1.000, và faithfulness đạt 1.000. Answer relevancy vẫn ở mức trung bình, khoảng 0.73, cho thấy còn dư địa cải thiện ở cách sinh và trình bày câu trả lời.

Mục lục

I.	Tổng quan báo cáo	1
II.	Định nghĩa bài toán	3
III.	Dữ liệu và nguồn tri thức sử dụng	3
IV.	Baseline: Hệ thống RAG cơ bản	4
IV.1.	Thành phần chính của baseline	5
IV.2.	Hạn chế của baseline	5
V.	Upgrade: Các cải tiến đã triển khai	7
V.1.	Từ giới hạn dữ liệu đến ingestion theo cấu trúc Notion	7
V.2.	Từ chunking tuyến tính đến chunking có cấu trúc	8
V.3.	Từ index tạm thời đến đồng bộ và lưu trữ bền vững	8
V.4.	Hybrid RRF	9
V.5.	Reranking và MMR	10
V.6.	Grounded generation và RAGAS evaluation	10
VI.	Kiến trúc hệ thống sau nâng cấp	11
VI.1.	Các module chính	12
VII.	Luồng xử lý chính	12
VIII.	Công cụ và công nghệ sử dụng	13
IX.	Cấu trúc mã nguồn và hướng dẫn chạy	14
IX.1.	Cài đặt và cấu hình	14
IX.2.	Chạy ứng dụng	15
IX.3.	Chạy kiểm thử và đánh giá	15
X.	Thực nghiệm và đánh giá	15
XI.	Phân tích lỗi và hạn chế	16
XII.	Kết luận và hướng phát triển	17
XIII.	Tài liệu tham khảo	18
	Phụ lục	19

II. Định nghĩa bài toán

Bài toán chính của project là **retrieval-augmented question answering** trên tài liệu học tập. Hệ thống nhận câu hỏi của người dùng, truy xuất các đoạn tài liệu liên quan từ kho tri thức đã index, sau đó dùng mô hình sinh ngôn ngữ để tạo câu trả lời dựa trên ngữ cảnh truy xuất được.

- **Đầu vào của hệ thống:** câu hỏi tiếng Việt hoặc tiếng Anh của người dùng, file PDF được upload từ giao diện Streamlit, và dữ liệu bài học từ Notion database/page content.
- **Đầu ra mong muốn:** câu trả lời tiếng Việt bám sát tài liệu, có nguồn trích dẫn, kèm thông tin metadata như tài liệu, trang PDF, tiêu đề Notion hoặc đường dẫn Notion nếu có.
- **Loại bài toán AI:** RAG, semantic search, keyword retrieval, reranking và grounded answer generation.
- **Tiêu chí thành công:** truy xuất đúng context, hạn chế trả lời ngoài tài liệu, có citation rõ ràng, demo chạy được, và có đánh giá bằng metric.

Về mặt sản phẩm, hệ thống cần phục vụ người học muốn hỏi nhanh nội dung trong tài liệu Module 1. Về mặt kỹ thuật, hệ thống cần quản lý nhiều nguồn dữ liệu, giữ metadata khi index, truy xuất ổn định và sinh câu trả lời có căn cứ.

III. Dữ liệu và nguồn tri thức sử dụng

Project sử dụng hai nhóm nguồn tri thức chính. Nhóm thứ nhất là các file PDF do người dùng upload từ giao diện Streamlit. Nhóm thứ hai là Notion database và nội dung lesson pages được đồng bộ thông qua Notion API.

Với PDF, hệ thống cần xử lý văn bản thông thường, bố cục trang, bảng và các trang hoặc ảnh cần OCR. Với Notion, hệ thống cần đọc metadata của từng bài học, lấy block tree, parse các block như heading, paragraph, list, code, equation, table và image. Sau đó toàn bộ nội dung được chuẩn hóa thành các document/chunk có metadata nhất quán trước khi đưa vào vector store.

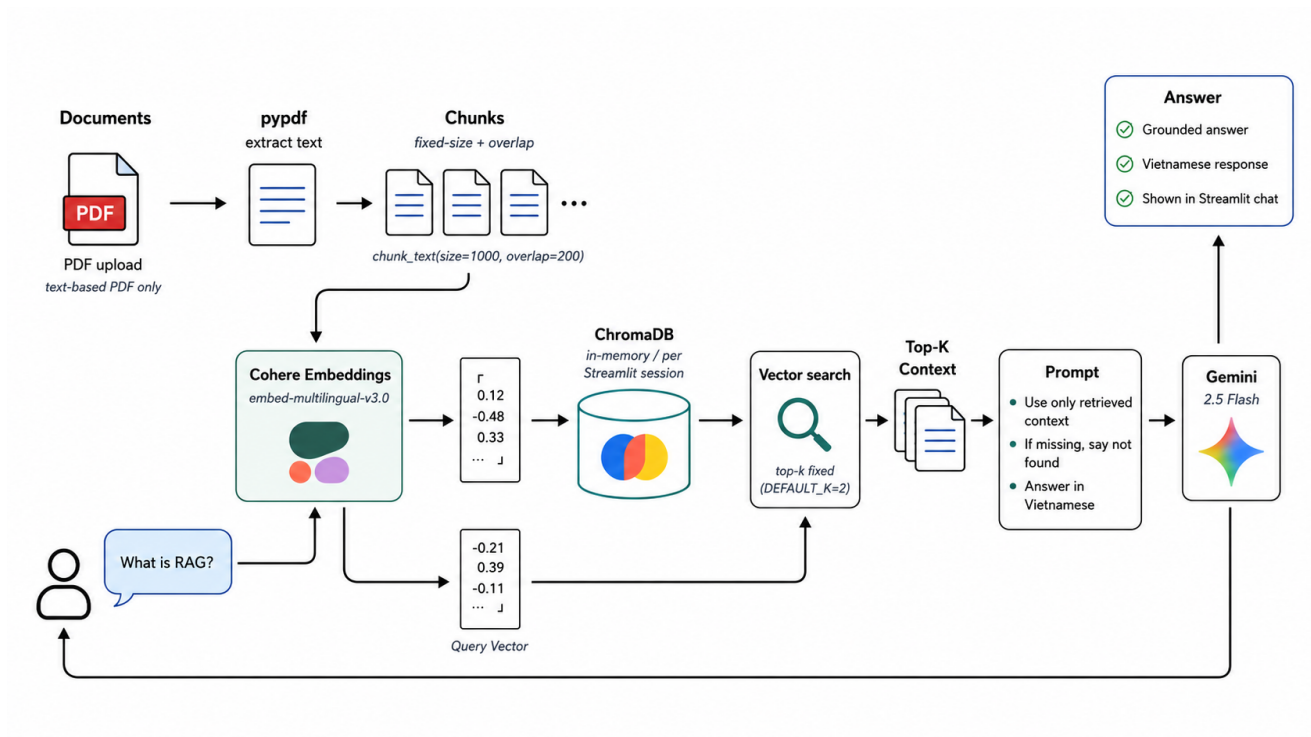
Nhóm metadata	Trường thông tin tiêu biểu
Metadata chung	source_type, source_granularity, chunk_id, block_types.
PDF	source_file, document_id, page_number, page_count, bbox, table_index, ocr_provider.
Notion	page_id, title, week, date, module, lecturer, label, heading_path, notion_url.

Bảng 1: Các nhóm metadata chính trong hệ thống

Các giới hạn chính của dữ liệu nằm ở chất lượng file đầu vào. OCR phụ thuộc chất lượng ảnh, độ rõ của trang scan và quota API. Bảng PDF có bố cục phức tạp có thể cần kiểm tra thủ công. Với Notion, các file ảnh hoặc URL tạm thời có thể hết hạn hoặc không truy cập được khi gọi OCR.

IV. Baseline: Hệ thống RAG cơ bản

Baseline được giữ trong file `baseline/baseline_chatbot_app.py` và đóng vai trò phiên bản tham chiếu. Baseline thể hiện luồng tối thiểu của một ứng dụng RAG trên PDF: người dùng upload PDF, hệ thống trích xuất text, chia đoạn, tạo embedding, lưu vector và trả lời câu hỏi dựa trên các đoạn liên quan.



Hình 1: Baseline Architecture.

Luồng xử lý baseline

```

PDF upload
-> pypdf text extraction
-> fixed-size chunking
-> Cohere embed-multilingual-v3.0
-> in-memory ChromaDB
-> vector search with fixed top-k
-> Gemini generation
-> Streamlit answer display

```

IV.1. Thành phần chính của baseline

Thành phần	Vai trò
Streamlit UI	Giao diện upload PDF, xử lý tài liệu, nhập câu hỏi và hiển thị lịch sử chat.
pypdf	Trích xuất text trực tiếp từ PDF.
chunk_text()	Chia văn bản thành các chunk cố định với overlap.
Cohere embed-multilingual-v3.0	Sinh embedding 1024 chiều cho chunk và câu hỏi thông qua Cohere API.
ChromaDB in-memory	Lưu vector trong phiên chạy hiện tại.
Gemini 2.5 Flash	Sinh câu trả lời tiếng Việt dựa trên context được truy xuất thông qua Gemini API.
requests	Gọi trực tiếp Cohere API và Gemini REST API.

Bảng 2: Các thành phần chính của baseline

IV.2. Hạn chế của baseline

Baseline đủ để minh họa nguyên lý RAG ở mức tối thiểu, nhưng còn nhiều điểm yếu khi áp dụng cho tài liệu học tập thật, đặc biệt là tài liệu có cấu trúc, có nhiều định dạng và cần kiểm chứng nguồn. Các hạn chế chính gồm:

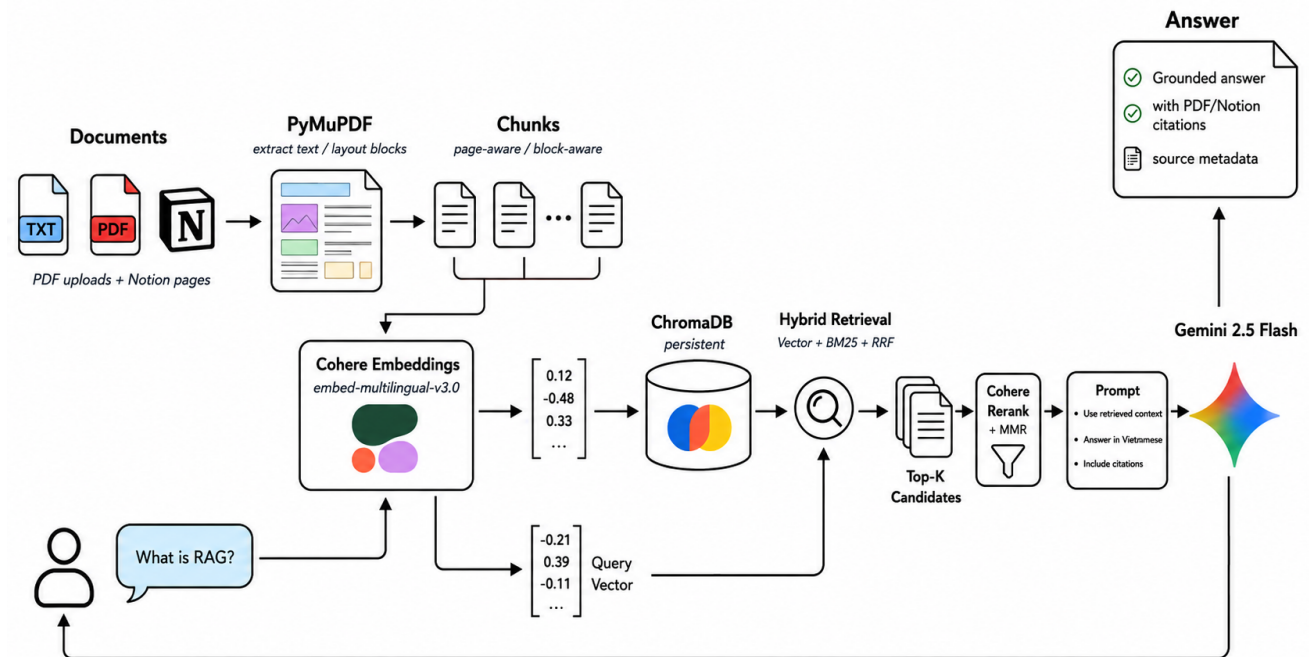
- Chunking còn đơn giản, chia văn bản theo độ dài cố định nên dễ cắt đứt ý nghĩa của một đoạn học, ví dụ tách rời định nghĩa, công thức, ví dụ hoặc phần giải thích đi kèm.
- Hệ thống chưa hiểu cấu trúc tài liệu như page, heading, block, code, table và equation. Vì vậy context truy xuất có thể thiếu tiêu đề, mất quan hệ giữa các phần hoặc làm phẳng nội dung có cấu trúc thành text tuyến tính.
- Chỉ xử lý PDF dạng text bằng `pypdf`; bảng, ảnh, trang scan, hình minh họa và layout phức tạp dễ bị mất thông tin.
- Citation còn yếu vì metadata ít. Chunk không bảo toàn đầy đủ nguồn, trang, heading hoặc URL nên câu trả lời khó được kiểm chứng lại từ tài liệu gốc.
- ChromaDB chạy in-memory nên dữ liệu index bị mất sau khi restart.
- Retrieval chỉ dùng vector search với top-k cố định. Cách này phù hợp với câu hỏi ngữ nghĩa tổng quát, nhưng chưa tốt cho câu hỏi cần từ khóa chính xác, tên hàm, thuật ngữ, tiêu đề bài học hoặc chi tiết xuất hiện ít.
- Không có rerank và MMR nên context có thể bị trùng lặp, nhiều hoặc thiếu đa dạng. Khi nhiều chunk gần giống nhau cùng được chọn, phần generation nhận ít thông tin bổ sung hơn dù top-k vẫn đủ số lượng.

- Không hỗ trợ Notion nên chưa khai thác được database bài học thật, lesson pages, metadata tuần học và liên kết nội dung đang được quản lý trong Notion.
- Không có cơ chế đồng bộ nên mỗi lần dữ liệu thay đổi phải index lại thủ công, khó theo dõi trang mới, trang đã sửa hoặc trang bị xóa.
- Không có evaluation set và metric định lượng nên khó biết hệ thống tốt hay kém ở khâu nào: retrieval, context selection hay generation.
- Chưa có test tự động cho các module chính, khiến việc mở rộng ingestion, chunking hoặc retrieval dễ phát sinh lỗi hồi quy.
- Error handling còn tối giản, dễ gây khó hiểu khi thiếu API key, lỗi API hoặc PDF rỗng.

V. Upgrade: Các cải tiến đã triển khai

Bản nâng cấp trong `upgrade_new/` được thiết kế trực tiếp từ các điểm yếu của baseline. Thay vì chỉ bổ sung tính năng rời rạc, phần upgrade đi theo mạch hệ thống: mở rộng nguồn tri thức, bảo toàn cấu trúc tài liệu khi chunking, lưu index bền vững, cải thiện retrieval bằng nhiều tín hiệu, chọn lọc context bằng reranking và MMR, kiểm soát generation bằng citation, và đo lường chất lượng bằng testset.

Mục tiêu kỹ thuật của bản nâng cấp là chuyển baseline từ một demo PDF RAG tối giản thành một hệ thống RAG có thể dùng cho tài liệu học tập thật. Hệ thống mới cần đọc được cả PDF và Notion, giữ được metadata để truy vết nguồn, giảm nhiễu trong context, trả lời bám tài liệu, đồng thời có cơ chế đánh giá để biết cải tiến nằm ở đâu.



Hình 2: Upgrade Architecture.

V.1. Từ giới hạn dữ liệu đến ingestion theo cấu trúc Notion

Baseline chỉ đọc PDF text bằng `pypdf`, nên thông tin trong bảng, ảnh, trang scan hoặc layout phức tạp dễ bị bỏ qua. Điều này làm giảm chất lượng context ngay từ đầu: nếu kiến thức không được nạp vào index, retrieval và generation phía sau không thể khôi phục lại.

Vì vậy bản nâng cấp thay tầng ingestion bằng các loader chuyên biệt hơn. Với PDF, hệ thống dùng `PyMuPDF` để trích xuất layout blocks theo trang, dùng `pdfplumber` cho bảng khi phù hợp, và hỗ trợ Gemini Vision OCR cho trang scan hoặc ảnh nhúng. Khi OCR hoặc table extraction lỗi, pipeline giảm cấp xử lý thay vì dùng toàn bộ quá trình index.

Với Notion, bản nâng cấp không coi nội dung như một khối văn bản phẳng. Notion database được xem là lớp quản lý tri thức cấp cao: mỗi row thường đại diện cho một lesson page, các property lưu metadata học tập, còn nội dung thật nằm trong block tree của page. Do đó ingestion phải ánh xạ đúng ba tầng: database property, page identity và block-level content. Cách ánh xạ này giúp hệ thống giữ được quan hệ giữa bài học, tiêu đề, block, bảng, code, equation, image và URL nguồn.

Cấu trúc trong Notion	Ánh xạ trong hệ thống RAG
Database row / page	Một đơn vị tài liệu gốc, có <code>page_id</code> , <code>notion_url</code> và trạng thái đồng bộ.
Page properties	Metadata như <code>title</code> , <code>week</code> , <code>module</code> , <code>lecturer</code> , <code>label</code> , <code>date</code> .
Block tree	Chuỗi content units theo thứ tự xuất hiện, giữ <code>block_id</code> , <code>block_type</code> và quan hệ cha-con.
Heading blocks	<code>heading_path</code> để bảo toàn ngữ cảnh tiêu đề khi chunking và citation.
Code, table, equation, image	Nội dung có kiểu riêng; table/code/equation được chuẩn hóa thành text có cấu trúc, image có thể qua Gemini Vision OCR khi bật.

Bảng 3: Ánh xạ cấu trúc Notion sang document/chunk trong hệ thống

Nhờ cách mapping này, nguồn tri thức không còn giới hạn ở file upload. Hệ thống khai thác được kho bài học đang quản lý trong Notion, đồng thời mỗi chunk sau indexing vẫn truy ngược được về page, heading và loại block ban đầu.

V.2. Từ chunking tuyến tính đến chunking có cấu trúc

Baseline chia văn bản theo độ dài cố định, nên một chunk có thể bị cắt giữa định nghĩa, ví dụ, bảng hoặc đoạn giải thích. Hạn chế này làm context thiếu mạch và khiến câu trả lời dễ chung chung hoặc thiếu căn cứ cụ thể.

Bản nâng cấp chuyển sang chunking có nhận thức cấu trúc. Với PDF, chunker giữ quan hệ theo trang; với Notion, chunker giữ quan hệ theo block; với cả hai nguồn, heading path được dùng để bảo toàn ngữ cảnh tiêu đề. Khi nội dung dài, hệ thống dùng recursive split theo đoạn hoặc câu thay vì cắt cơ học ở một vị trí bất kỳ.

Metadata cũng được làm giàu trong quá trình này, gồm source type, page number, title, week, module, heading path và Notion URL khi có. Lợi ích trực tiếp là mỗi chunk không chỉ là text, mà còn mang theo ngữ cảnh để hiển thị citation, debug lỗi retrieval và phân tích kết quả đánh giá.

V.3. Từ index tạm thời đến đồng bộ và lưu trữ bền vững

Baseline dùng ChromaDB in-memory, vì vậy index bị mất sau khi ứng dụng restart. Với dữ liệu Notion, baseline cũng không có cách phát hiện nội dung mới hoặc nội dung thay đổi. Điều này không phù hợp với một hệ thống học tập có dữ liệu được cập nhật liên tục.

Bản nâng cấp dùng persistent ChromaDB collection để giữ index qua nhiều phiên chạy. Riêng với Notion, hệ thống hỗ trợ full sync và incremental sync. Local sync state lưu hash để phát hiện trang mới, trang thay đổi hoặc trang bị xóa; tùy chọn Content Hash writeback cho phép ghi

hash ngược về Notion sau khi index thành công.

Cách triển khai này làm giảm chi phí index lại toàn bộ dữ liệu, đồng thời giúp quá trình cập nhật có thể kiểm soát được. Thay vì coi ingestion là thao tác một lần, hệ thống biến nó thành một pipeline đồng bộ có trạng thái.

V.4. Hybrid RRF

Baseline chỉ dùng vector search với top-k cố định. Cách này có thể tìm được đoạn gần nghĩa, nhưng yếu với câu hỏi có keyword chính xác, tên hàm, tên bài học hoặc thuật ngữ ngắn. Do đó bản nâng cấp tách retrieval thành hai nhánh: vector retrieval để bắt quan hệ ngữ nghĩa và BM25 keyword retrieval để bắt trùng khớp từ khóa.

Ở nhánh vector, câu hỏi q và chunk d được biểu diễn bằng embedding Cohere. Mức gần nhau có thể diễn giải bằng cosine similarity:

$$S_{\text{vec}}(q, d) = \cos(e_q, e_d) = \frac{e_q \cdot e_d}{\|e_q\| \|e_d\|}, \quad (1)$$

Trong đó: e_q là embedding của câu hỏi e_d là embedding của chunk S_{vec} càng lớn thì hai biểu diễn càng gần nhau về ngữ nghĩa. Trong cài đặt hiện tại, vector retrieval được thực hiện qua ChromaDB persistent collection với query embedding từ Cohere `embed-multilingual-v3.0`.

Ở nhánh keyword, hệ thống xây BM25 index cục bộ từ các chunk đã lưu. Với query term t , chunk d , số lần xuất hiện $f(t, d)$, độ dài chunk $|d|$, độ dài trung bình avgdl , công thức BM25 được dùng ở dạng:

$$S_{\text{BM25}}(q, d) = \sum_{t \in q} \text{IDF}(t) \frac{f(t, d)(k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)}. \quad (2)$$

Trong đó: $k_1 = 1.5$ và $b = 0.75$ theo cài đặt của `KeywordStore`. Tokenizer hỗ trợ tiếng Việt và sinh thêm biến thể bỏ dấu, nên các câu hỏi không dấu vẫn có cơ hội khớp với tài liệu có dấu.

Hai danh sách kết quả không được cộng trực tiếp bằng raw score vì vector score và BM25 score có thang đo khác nhau. Hệ thống dùng Reciprocal Rank Fusion (RRF) để hợp nhất theo thứ hạng:

$$S_{\text{RRF}}(d) = \sum_{r \in \mathcal{R}} \frac{1}{k_{\text{RRF}} + \text{rank}_r(d)}. \quad (3)$$

Ở đây $\mathcal{R} = \{\text{vector}, \text{keyword}\}$, $\text{rank}_r(d)$ là thứ hạng 1-based của chunk d trong nhánh retrieval r , và chunk không xuất hiện trong một nhánh thì không đóng góp điểm từ nhánh đó. Hệ số k_{RRF} làm giảm ảnh hưởng cực đoan của các vị trí đầu; cài đặt hiện tại dùng `RRF_K=60`. Hệ thống lấy tối đa `HYBRID_CANDIDATE_K=30` ứng viên mỗi nhánh, hợp nhất theo id, gán `vector_rank`, `keyword_rank`, `rrf_score`, rồi sắp xếp giảm dần theo `rrf_score`.

Lợi ích của Hybrid RRF là retrieval không phụ thuộc vào một tín hiệu duy nhất. Vector search xử lý câu hỏi ngữ nghĩa, BM25 hỗ trợ keyword chính xác, còn RRF hợp nhất hai nguồn theo thứ hạng nên ổn định hơn khi raw score giữa các nhánh không đồng nhất.

V.5. Reranking và MMR

Sau Hybrid RRF, danh sách ứng viên vẫn có thể chứa chunk gần đúng nhưng chưa phải context tốt nhất cho câu hỏi. Vì vậy bản nâng cấp thêm Cohere rerank để đánh giá lại từng cặp (q, d) ở mức semantic matching trực tiếp hơn:

$$S_{\text{rerank}}(q, d) = f_{\theta}(q, d), \quad (4)$$

trong đó f_{θ} là rerank model, q là câu hỏi và d là văn bản ứng viên. Trong hệ thống hiện tại, provider là Cohere khi `COHERE_API_KEY` tồn tại, model mặc định là `rerank-v3.5`. Trước khi gửi rerank, mỗi candidate được định dạng kèm metadata như title, source type, heading path, page number và phần text rút gọn. Cohere trả về `relevance_score`; hệ thống gán `rerank_score`, `rerank_rank` và dùng danh sách đã sắp xếp lại cho bước chọn context.

Rerank giúp tăng độ liên quan, nhưng nếu chỉ lấy top-k sau rerank thì context có thể bị trùng ý. Vì vậy hệ thống dùng Maximal Marginal Relevance (MMR) để cân bằng giữa liên quan và đa dạng:

$$d^* = \arg \max_{d_i \in C \setminus S} \left[\lambda \text{Rel}(q, d_i) - (1 - \lambda) \max_{d_j \in S} \text{Sim}(d_i, d_j) \right]. \quad (5)$$

Trong đó C là tập candidate sau retrieval/rerank, S là tập context đã chọn, $\text{Rel}(q, d_i)$ là điểm liên quan đã chuẩn hóa, $\text{Sim}(d_i, d_j)$ là độ giống nhau giữa hai chunk, và λ điều chỉnh mức ưu tiên giữa relevance và diversity. Cài đặt hiện tại dùng `MMR_LAMBDA=0.65`, nghĩa là ưu tiên relevance nhưng vẫn phạt các chunk quá giống nhau.

Trong implementation, Rel được lấy theo thứ tự ưu tiên từ rerank score, RRF score, vector score và keyword score, rồi chuẩn hóa về khoảng $[0, 1]$. Độ giống nhau Sim được tính bằng Jaccard similarity trên token:

$$\text{Sim}(d_i, d_j) = \frac{|T(d_i) \cap T(d_j)|}{|T(d_i) \cup T(d_j)|}, \quad (6)$$

trong đó $T(d)$ là tập token của chunk d . Ngưỡng `MMR_MIN_TEXT_SIMILARITY=0.82` được dùng để nhận diện ứng viên gần trùng. Kết quả cuối cùng là tập context vừa liên quan vừa giảm lặp, phù hợp hơn cho bước generation.

V.6. Grounded generation và RAGAS evaluation

Trong baseline, câu trả lời được sinh từ context truy xuất nhưng citation còn yếu và không có đánh giá định lượng. Vì vậy khó xác định lỗi đến từ việc truy xuất sai context, context thiếu thông tin hay model sinh câu trả lời chưa đúng trọng tâm.

Bản nâng cấp dùng Gemini cho generation nhưng ràng buộc prompt theo retrieved context. Sau MMR, hệ thống có tập context $C_q = \{d_1, \dots, d_K\}$. Câu trả lời được mô hình hóa như:

$$a = G_{\theta}(q, C_q, I), \quad (7)$$

trong đó a là câu trả lời, G_{θ} là generation model, q là câu hỏi, C_q là context đã chọn và I là instruction yêu cầu chỉ trả lời dựa trên context. Giao diện hiển thị citation và metadata nguồn để người dùng kiểm tra lại câu trả lời. Trong cài đặt hiện tại, provider mặc định là Gemini,

model mặc định là **gemini-2.5-flash**; hệ thống cũng có fallback theo cấu hình nếu generation provider thay đổi.

Để đánh giá offline, hệ thống dùng testset dạng JSONL và có thể tích hợp RAGAS. Với faithfulness, câu trả lời được tách thành các claim c_i ; metric có thể diễn giải bằng tỷ lệ claim được context hỗ trợ:

$$\text{Faithfulness} = \frac{|\{c_i \in \text{Claims}(a) : \text{Supported}(c_i, C_q)\}|}{|\text{Claims}(a)|}. \quad (8)$$

Metric này trả lời câu hỏi: câu trả lời có bám vào context được truy xuất hay không.

Answer relevancy đo mức câu trả lời đi đúng trọng tâm câu hỏi. Một cách diễn giải thông dụng trong RAGAS là sinh lại các câu hỏi $\hat{q}_i$ từ answer, rồi so sánh với câu hỏi gốc:

$$\text{AnswerRel}(q, a) = \frac{1}{m} \sum_{i=1}^m \cos(E(q), E(\hat{q}_i)). \quad (9)$$

Trong đó $E(\cdot)$ là embedding function, m là số câu hỏi được sinh lại, và giá trị càng cao thì answer càng tập trung vào nhu cầu truy vấn ban đầu.

Context precision đo mức các context liên quan được xếp cao trong top-k. Với $\text{rel}_k \in \{0, 1\}$ thể hiện chunk ở vị trí k có liên quan hay không:

$$\text{ContextPrecision@K} = \frac{1}{\sum_{k=1}^K \text{rel}_k} \sum_{k=1}^K \left(\frac{\sum_{i=1}^k \text{rel}_i}{k} \right) \text{rel}_k. \quad (10)$$

Context recall đo mức context truy xuất bao phủ đủ thông tin trong ground truth g :

$$\text{ContextRecall} = \frac{|\{s_i \in \text{Statements}(g) : \text{Supported}(s_i, C_q)\}|}{|\text{Statements}(g)|}. \quad (11)$$

Trong hệ thống hiện tại, script evaluation chạy so sánh vector-only và Hybrid RRF + Cohere Rerank + MMR, sinh báo cáo JSON/CSV/Markdown. Nhờ đó phần upgrade không chỉ dừng ở cảm nhận demo tốt hơn: kết quả ở mục X cho thấy context precision, context recall và faithfulness tăng rõ, còn answer relevancy vẫn là điểm cần tiếp tục tối ưu bằng prompt và chiến lược trình bày câu trả lời.

VI. Kiến trúc hệ thống sau nâng cấp

Kiến trúc sau nâng cấp có thể tóm tắt bằng pipeline sau:

Pipeline tổng quan

```
PDF / Notion
-> Loader / Parser
-> Structure-aware Chunker
-> Cohere Embeddings
```

```

-> Persistent ChromaDB
-> Hybrid Retriever: Vector + BM25 + RRF
-> Cohere Rerank
-> MMR Context Selection
-> Gemini Grounded Generation
-> Streamlit UI with citations

```

VI.1. Các module chính

Module	Vai trò
app.py	Giao diện Streamlit, upload/index PDF, Notion sync, chat, citation và debug display.
src/loaders/pdf_loader.py	Trích xuất text/layout/image units từ PDF, tích hợp OCR và table extraction.
src/loaders/notion_loader.py	Query Notion database/page, parse metadata, đọc và ghi Content Hash.
src/loaders/notion_block_parser.py	Chuyển block tree của Notion thành structured content units.
src/chunker.py	Chia nội dung thành chunk có nhận thức page, block và heading.
src/embeddings.py	Gọi Cohere multilingual embeddings.
src/vector_store.py	Quản lý persistent ChromaDB collection.
src/retriever.py	Vector retrieval, keyword retrieval và hybrid RRF.
src/reranker.py	Cohere rerank và MMR selection.
src/rag_chain.py	Tạo grounded context và gọi model sinh câu trả lời.
src/sync/notion_sync.py	Full/incremental Notion sync, local sync state và hash checks.
src/evaluation.py	Sinh báo cáo đánh giá offline và tích hợp RAGAS khi được bật.

Bảng 4: Các module chính trong `upgrade_new/`

VII. Luồng xử lý chính

Luồng xử lý của hệ thống sau nâng cấp có thể chia thành hai pha liên tục: pha xây dựng chỉ mục và pha hỏi đáp/đánh giá. Cả hai pha dùng chung cùng một biểu diễn tài liệu: nội dung được chuẩn hóa thành content units, sau đó thành chunk có metadata đầy đủ để phục vụ retrieval, citation và phân tích lỗi.

Ở pha xây dựng chỉ mục, người dùng có thể đưa dữ liệu vào từ PDF hoặc Notion. Với PDF, hệ thống đọc từng trang bằng PyMuPDF, trích xuất block văn bản theo layout, nhận diện bảng bằng pdfplumber khi phù hợp và gọi OCR/Vision cho trang scan hoặc ảnh nếu `ENABLE_OCR=true`. Với Notion, hệ thống query database, đọc page properties, fetch block tree của từng lesson page, parse paragraph/heading/list/code/table/equation/image và giữ lại `heading_path` cùng `notion_url`. Sau bước ingestion, các content units được chunk, embedding bằng Cohere và ghi vào persistent ChromaDB.

Riêng với Notion, quá trình index được gắn với cơ chế đồng bộ. Full sync dùng khi cần nạp lại toàn bộ database. Incremental sync dùng local sync state và content hash để phát hiện page mới, page đã thay đổi hoặc page bị xóa, từ đó hạn chế re-index toàn bộ dữ liệu. Nếu bật `ENABLE_NOTION_HASH_WRITE=true`, hệ thống có thể ghi `Content Hash` về Notion sau khi index thành công để hỗ trợ kiểm tra trạng thái đồng bộ.

Ở pha hỏi đáp, câu hỏi người dùng được embedding và đưa vào retrieval pipeline. Hệ thống truy xuất song song bằng vector search và BM25, hợp nhất ứng viên bằng RRF, rerank bằng Cohere nếu bật, rồi chọn context cuối bằng MMR. Gemini nhận câu hỏi cùng context đã chọn và sinh câu trả lời theo ràng buộc grounded generation. Giao diện Streamlit hiển thị answer, citation, metadata nguồn và thông tin debug khi cần.

Ở pha đánh giá offline, script evaluation đọc testset JSONL, chạy từng cấu hình pipeline và lưu kết quả ra JSON, CSV hoặc Markdown. Khi bật RAGAS, hệ thống chấm faithfulness, answer relevancy, context precision và context recall. Vì vậy cùng một pipeline có thể được kiểm tra theo cả hai hướng: chạy demo tương tác trên giao diện và đo chất lượng định lượng trên testset cố định.

VIII. Công cụ và công nghệ sử dụng

Công nghệ	Vai trò trong project
Python	Ngôn ngữ chính để xây dựng pipeline ingestion, retrieval, generation, sync và evaluation.
Streamlit	Giao diện demo cho upload PDF, đồng bộ Notion, hỏi đáp và hiển thị citation.
PyMuPDF	Trích xuất layout blocks và nội dung theo trang từ PDF.
pdfplumber	Trích xuất bảng từ PDF khi có cấu trúc bảng phù hợp.
Notion API	Đồng bộ database metadata và nội dung lesson pages.
ChromaDB	Lưu trữ vector persistent cho các chunk đã index.
Cohere Embeddings	Sinh multilingual embeddings cho tài liệu và query.
Cohere Rerank	Sắp xếp lại các candidate context sau bước retrieval ban đầu.
Gemini	OCR/Vision tùy chọn và sinh câu trả lời grounded.
RAGAS	Đánh giá faithfulness, answer relevancy, context precision và context recall.
pytest	Kiểm thử các module chunking, loading, sync, retrieval, reranking và RAG chain.

Bảng 5: Công cụ và công nghệ chính

IX. Cấu trúc mã nguồn và hướng dẫn chạy

Repository được tổ chức để tách rõ baseline tham chiếu và bản nâng cấp chính. Phần báo cáo này tập trung vào `upgrade_new/`, là workspace hoàn chỉnh nhất của project.

Cấu trúc thư mục chính

```
project/
|-- baseline/
|   '-- baseline_chatbot_app.py
|
|-- upgrade_new/
|   |-- app.py
|   |-- requirements.txt
|   |-- requirements-eval.txt
|   |-- src/
|   |-- tests/
|   |-- docs/
|   |-- eval/
|   '-- scripts/
|
|-- tests/
|-- docs/
'-- Report_Conquer/
```

IX.1. Cài đặt và cấu hình

Cài đặt dependencies

```
pip install -r upgrade_new/requirements.txt
copy upgrade_new/.env.example upgrade_new/.env
```

Các biến môi trường chính gồm:

- COHERE_API_KEY, GEMINI_API_KEY, NOTION_TOKEN, NOTION_DATABASE_ID.
- Tùy chọn: ENABLE_OCR=true, ENABLE_NOTION_HASH_WRITE=true, ENABLE_MMR=true.

IX.2. Chạy ứng dụng

Chạy Streamlit app

```
streamlit run upgrade_new/app.py --server.port 8503
```

Sau khi chạy, giao diện được mở tại <http://localhost:8503/>. Sidebar hiển thị trạng thái OCR/Vision, Notion hash write, retrieval mode, rerank và MMR.

IX.3. Chạy kiểm thử và đánh giá

Chạy test và evaluation

```
python -m pytest tests upgrade_new/tests

python upgrade_new/scripts/run_ragas_eval.py
--testset upgrade_new/eval/testset.grounded.jsonl
--compare-pipelines --formats json,csv,md
```

X. Thực nghiệm và đánh giá

Đánh giá hiện tại được thực hiện trên testset grounded tại `upgrade_new/eval/testset.grounded.jsonl`. Testset gồm 6 câu hỏi, mỗi câu có ground truth/reference để phục vụ đánh giá. Hai pipeline được so sánh là vector-only và Hybrid RRF + Cohere Rerank + MMR.

Các metric được sử dụng gồm:

- **Faithfulness:** mức độ câu trả lời được hỗ trợ bởi context truy xuất.
- **Answer relevancy:** mức độ câu trả lời đúng trọng tâm câu hỏi.
- **Context precision:** mức độ context truy xuất liên quan và được xếp hạng tốt.
- **Context recall:** mức độ context truy xuất bao phủ đủ thông tin cần thiết trong ground truth.

Pipeline	Faithfulness	Answer relevancy	Context precision	Context recall
Vector only	0.958	0.725	0.972	0.750
Hybrid RRF + Cohere Rerank + MMR	1.000	0.729	1.000	1.000

Bảng 6: Kết quả RAGAS trên testset grounded

Kết quả cho thấy pipeline nâng cấp cải thiện rõ chất lượng truy xuất. Context recall tăng từ 0.750 lên 1.000, cho thấy hệ thống tìm được đầy đủ hơn các đoạn cần thiết để trả lời. Context precision tăng lên 1.000, phản ánh việc kết hợp RRF, rerank và MMR giúp lọc context liên quan tốt hơn. Faithfulness đạt 1.000, cho thấy câu trả lời trong lần đánh giá này bám sát context.

Answer relevancy chỉ tăng nhẹ từ 0.725 lên 0.729. Điều này cho thấy cải tiến retrieval chưa tự động giải quyết toàn bộ chất lượng câu trả lời cuối cùng. Phần generation và prompt vẫn cần được tối ưu để câu trả lời ngắn hơn, đúng trọng tâm hơn và phù hợp hơn với ground truth.

XI. Phân tích lỗi và hạn chế

Mặc dù bản nâng cấp đã cải thiện đáng kể so với baseline, hệ thống vẫn còn một số hạn chế cần ghi nhận:

- Testset đánh giá hiện mới có 6 câu hỏi, chưa đủ lớn để đại diện toàn bộ nội dung học tập và các kiểu câu hỏi khác nhau.
- Answer relevancy còn ở mức khoảng 0.73; câu trả lời có thể còn dài, bao quát hoặc chưa khớp hoàn toàn với trọng tâm câu hỏi.
- OCR/Vision phụ thuộc vào chất lượng ảnh, khả năng truy cập file và quota của Gemini API.
- Bảng PDF có bố cục phức tạp vẫn cần kiểm tra thủ công sau khi trích xuất bằng pdfplumber.
- Hệ thống chưa triển khai chat memory, HyDE, query decomposition và follow-up question rewriting.
- Ứng dụng hiện phù hợp cho demo local hoặc submission, chưa được harden cho triển khai production nhiều người dùng.

Các hạn chế này không làm mất giá trị của bản nâng cấp, nhưng cần được ghi nhận để đánh giá đúng phạm vi hiện tại của project và định hướng phát triển tiếp theo.

XII. Kết luận và hướng phát triển

Project đã xây dựng được một hệ thống RAG có khả năng chạy được, có nhiều nguồn dữ liệu, có pipeline index/truy xuất/sinh câu trả lời, có citation và có đánh giá định lượng. So với baseline, bản nâng cấp cải thiện ở các điểm chính: xử lý PDF tốt hơn, bổ sung Notion sync, chunking có nhận thức cấu trúc, vector store persistent, hybrid retrieval, reranking, MMR, citation UI, test tự động và evaluation report.

Kết quả thực nghiệm cho thấy hướng nâng cấp retrieval là có cơ sở. Pipeline **Hybrid RRF + Cohere Rerank + MMR** đạt context precision, context recall và faithfulness ở mức 1.000 trên testset grounded hiện tại. Tuy nhiên, answer relevancy còn cần cải thiện, đặc biệt ở prompt, chiến lược sinh câu trả lời và quy mô bộ đánh giá.

Các hướng phát triển tiếp theo gồm:

- Mở rộng grounded testset lên nhiều loại câu hỏi hơn, gồm fact, summary, comparison, table, code và metadata questions.
- Bổ sung per-source recall để kiểm tra hệ thống có truy xuất đúng page, heading hoặc Notion source cần thiết hay không.
- Tối ưu prompt và answer style để tăng answer relevancy.
- Thử nghiệm HyDE, query decomposition và follow-up question rewriting.
- Bổ sung chat memory có kiểm soát để hỗ trợ hội thoại nhiều lượt.
- Kiểm thử thêm với PDF scan, PDF bảng phức tạp và Notion pages có ảnh hoặc bảng dài.
- Chuẩn hóa thêm quy trình deployment nếu cần đưa hệ thống ra môi trường nhiều người dùng.

XIII. Tài liệu tham khảo

- [1] TA Quốc Thái, *Video bài giảng M01W04 - Project 1.1: Web and Streamlit for Simple Deployment (Using NLP Libs)*, Tài liệu AI Việt Nam / AIO 2026, 2026.
- [2] Quân-Bảo Nguyễn, Mai-Anh Vũ, Quang-Vinh Đinh, *Tài liệu đọc: Hướng Dẫn Xây Dựng Ứng Dụng Web Phân Tích Điểm Số Học Sinh Đơn Giản với Streamlit*, Tài liệu AI Việt Nam / AIO 2026, 2026.
- [3] Nguyễn Quốc Thái, Hồ Quang Hiên, Đinh Quang Vinh, *Tài liệu đọc: Project 1.1: NLP Applications using Streamlit*, Tài liệu AI Việt Nam / AIO 2026, 2026.
- [4] TA Quốc Thái, *Video bài giảng M01W04 - Project 1.2: Using OpenAI API to build a chatbot quickly*, Tài liệu AI Việt Nam / AIO 2026, 2026.
- [5] Nguyễn Quốc Thái, Nguyễn Phúc Nguyên, Đinh Quang Vinh, *Tài liệu đọc: Project 1.2: Xây dựng Chatbot RAG hỏi đáp tài liệu*, Tài liệu AI Việt Nam / AIO 2026, 2026.
- [6] Streamlit, *Streamlit Documentation*, <https://docs.streamlit.io/>, 2026.
- [7] Chroma, *Chroma Docs: Introduction*, <https://docs.trychroma.com/docs/overview/introduction>, 2026.
- [8] Ragas, *Ragas Documentation*, <https://docs.ragas.io/en/stable/>, 2026.

Phụ lục

1. **GitHub Repository:** <https://github.com/AIVIETNAM-AIO-springtank/Tank-Tank-RAG-Bot-Learning-Assistant.git>
2. **Demo Video:** <https://youtu.be/4edPhX62Ph4>
3. **Demo App BASELINE:** <https://aio-2026-rag-learning-assistant-8eyegmdv2f2vdkewm8kccv.streamlit.app/>
4. **Demo App UPGRADE:** <https://tank-tank-rag-bot-learning-assistant-n69xd3y5fj8twkik8n3dtm.streamlit.app/>
5. **Testset:** Bộ test grounded hiện đặt tại `upgrade_new/eval/testset.grounded.jsonl`.
6. **Evaluation Reports:** Các báo cáo đánh giá nằm trong `upgrade_new/eval/reports/`, gồm Markdown, CSV và JSON.
7. **Model/API Providers:** Baseline dùng Cohere embeddings và Gemini generation; bản nâng cấp tiếp tục dùng Cohere embeddings/rerank và Gemini generation/OCR theo cấu hình môi trường.